FSH - Projet Système

Généré par Doxygen 1.9.1

Chapitre 1

AUTHORS

Nom Prenom Numéro étudiant Pseudo Gitlab

Dahmane Ilan 22115745 dahmane ilan Achirafi Amal 22111822 achirafi amal Dominique Zoé 22203393 Dominique martin

Chapitre 2

Projet : programmation d'un interpréteur de commandes

L3 Informatique - Système

Il est important de bien lire le sujet jusqu'au bout et de bien y réfléchir avant de se lancer dans la programmation du projet.

2.1 Sujet : <tt>fsh</tt>, un petit shell avec des boucles <tt>for</tt>

Le but du projet est de programmer un interpréteur de commandes (aka *shell*) interactif reprenant quelques fonctionnalités plus ou moins classiques des shells usuels : outre la possibilité d'exécuter toutes les commandes externes, `fsh` devra proposer quelques commandes internes, permettre la redirection des flots standard ainsi que les combinaisons par tube, adapter le prompt à la situation, et supporter les structures conditionnelles et un certain type particulier de boucles.

La gestion des tâches n'est pas requise : tous les processus lancés à partir de votre shell le seront en premier-plan.

Plus précisément, `fsh` doit respecter les prescriptions suivantes :

2.1.1 Commandes externes

`fsh` peut exécuter toutes les commandes externes, avec ou sans arguments, en tenant compte de la variable d'environnement `PATH`.

2.1.2 Commandes internes

`fsh` possède (au moins) les commandes internes listées ci-dessous. Elles ont comme valeur de retour 0 en cas de succès (à l'exception d'`exit`, naturellement), et 1 en cas d'échec.

2.1.2.0.1 <**tt**>**pwd**</**tt**> Affiche la référence physique absolue du répertoire de travail courant.

2.1.2.0.2 <**tt**>**cd [REF | -]**</**tt**> Change de répertoire de travail courant, qui devient le répertoire `REF` (s'il s'agit d'une référence valide), le précédent répertoire de travail si le paramètre est `-`, ou `$HOME` en l'absence de paramètre.

2.1.2.0.3 <**tt**>**exit [VAL]**</**tt**> Termine le processus `fsh` avec comme valeur de retour `VAL` (ou par défaut la valeur de retour de la dernière commande exécutée).

Lorsqu'il atteint la fin de son entrée standard (ie si l'utilisateur saisit `ctrl-D` en mode interactif), `fsh` se comporte comme si la commande interne `exit` (sans paramètre) avait été saisie.

2.1.2.0.4 <**tt**>**ftype REF**</**tt**> Affiche le type du fichier de référence `REF` (s'il s'agit d'une référence valide) : `directory`, `regular file` , `symbolic link`, `named pipe`, `other`.

2.1.3 Redirections vers des fichiers

`fsh` gère les redirections suivantes :

- `CMD < FIC` : redirection de l'entrée standard de la commande `CMD` sur le fichier (ordinaire, tube nommé...) `FIC`
- `CMD > FIC` : redirection de la sortie standard de la commande `CMD` sur le fichier (ordinaire, tube nommé...) `FIC` **sans écrasement** (ie, échoue si `FIC` existe déjà)
- `CMD >| FIC` : redirection de la sortie standard de la commande `CMD` sur le fichier (ordinaire, tube nommé...) `FIC` **avec écrasement** éventuel
- `CMD >> FIC` : redirection de la sortie standard de la commande `CMD` sur le fichier (ordinaire, tube nommé...) `FIC` **en concaténation**
- `CMD 2> FIC` : redirection de la sortie erreur standard de la commande `cmd` sur le fichier (ordinaire, tube nommé...) `FIC` **sans écrasement** (ie, échoue si `FIC` existe déjà)
- `CMD 2>| FIC` : redirection de la sortie erreur standard de la commande `CMD` sur le fichier (ordinaire, tube nommé...) `FIC` **avec écrasement** éventuel
- `CMD 2>> FIC` : redirection de la sortie erreur standard de la commande `CMD` sur le fichier (ordinaire, tube nommé...) `FIC` **en concaténation**

Les espaces de part et d'autre des symboles de redirection sont requis.

En cas d'échec d'une redirection, la ligne de commande saisie n'est pas exécutée, et la valeur de retour est 1.

2.1.4 Hiérarchie des lignes de commande

2.1.4.1 Commandes simples

Une **commande simple** est une ligne de commande exécutant une unique commande, interne ou externe, avec ou sans redirection.

2.1.4.2 Pipelines

Si `CMD_1` et `CMD_2` sont deux commandes simples, `CMD_1 | CMD_2` provoque l'exécution **simultanée** de `CMD_1` et `CMD_2`, avec redirection de la sortie standard de `CMD_1` et de l'entrée standard de `CMD_2` sur un même tube anonyme.

Un **pipeline** est soit une commande simple, soit une ligne de commande de la forme `CMD_1 | CMD_2 | ... | CMD_N`, avec les redirections additionnelles éventuelles :

- redirection de l'entrée standard de `CMD_1`
- redirection de la sortie standard de `CMD_N`
- redirection des sorties erreurs des `N` commandes.

Les espaces de part et d'autre du symbole `|` sont requis.

La valeur de retour est celle de `CMD_N`.

2.1.4.3 Commandes structurées

Une **commande structurée** est soit un pipeline, soit une ligne de commande d'une des formes suivantes :

2.1.4.3.1 <tt>**CMD_1 ; CMD_2 ; ... ; CMD_N**</tt> Exécute `CMD_1`, **puis** après la terminaison de celle-ci, `CMD_2`, et ainsi de suite jusqu'à `CMD_N`.

La valeur de retour est celle de `CMD_N`.

2.1.4.3.2 <tt>**if TEST { CMD }**</tt> Exécute le pipeline `TEST` puis, en cas de succès (*ie* si sa valeur de retour est 0), exécute la commande structurée `CMD` entre accolades.

La valeur de retour est celle de `CMD` si celle-ci est exécutée, et 0 sinon.

2.1.4.3.3 <tt>**if TEST { CMD_1 } else { CMD_2 }**</tt> Exécute le pipeline `TEST` puis, selon sa valeur de retour, exécute l'une ou l'autre des commandes structurées entre accolades.

La valeur de retour est celle de `CMD_1` ou de `CMD_2` selon le cas.

2.1.4.3.4 <tt>**for F in REP [-A] [-r] [-e EXT] [-t TYPE] [-p MAX] { CMD }**</tt> Sans option, exécute la commande structurée `CMD` pour chaque (référence de) fichier non caché du répertoire `REP`. `CMD` peut dépendre de la valeur de la variable de boucle `F`, désignée par `$F`. Par souci de simplification, les noms de variables sont limités à un caractère.

Effet des options :

- `-A` : étend la portée de la boucle aux fichiers cachés (sauf `.` et `..`);
- `-r` : parcours récursif de toute l'arborescence de racine `REP`;
- `-e EXT` : filtrage des résultats selon l'extension : les fichiers dont le nom de base ne se termine pas par `.EXT` sont ignorés; pour les autres fichiers, la variable de boucle `F` prend pour valeur la référence du fichier *amputée de son extension*, pour permettre des usages tels que `for F in . -e jpg { convert $F.jpg $F.png }`;
- `-t TYPE` : filtrage des résultats selon le type de fichier : `f` pour les fichiers ordinaires, `d` pour les répertoires, `l` pour les liens symboliques, `p` pour les tubes;
- `-p MAX` : permet le traitement en parallèle d'un maximum de `MAX` tours de boucle.

La valeur de retour est le **maximum** des valeurs de retour des différents tours de boucle.

Les espaces de part et d'autre des symboles `;`, `{` et `}` sont requis.

2.1.4.4 Autres lignes de commande

Les commandes structurées sont les lignes de commande les plus génériques que `fsh` doit savoir interpréter et exécuter. En particulier, il n'est pas demandé de pouvoir rediriger l'entrée ou la sortie d'une commande structurée, que ce soit vers un fichier ou dans un pipeline.

2.1.5 Gestion de la ligne de commande

`fsh` fonctionne en mode interactif : il affiche une invite de commande (*prompt*), lit la ligne de commande saisie par l'utilisateur, la découpe en mots selon les (blocs d')espaces, interprète les éventuels caractères spéciaux (`<`, `>`, `|`, `[`, `}`, `;`, `$`), puis exécute la ou les commandes correspondantes.

`fsh` ne fournit aucun mécanisme d'échappement; les arguments de la ligne de commande ne peuvent donc pas contenir de caractères spéciaux (espaces en particulier).

Il est fortement recommandé d'utiliser la bibliothèque `readline` pour simplifier la lecture de la ligne de commande : cette bibliothèque offre de nombreuses possibilités d'édition pendant la saisie de la ligne de commande, de gestion de l'historique, de complétion automatique... Sans entrer dans les détails, un usage basique de la fonction `char *readline (const char *prompt)` fournit déjà un résultat très satisfaisant :

```
char * ligne = readline("mon joli prompt $ ");
/* alloue et remplit ligne avec une version "propre" de la ligne saisie,
 * ie nettoyée des éventuelles erreurs et corrections (et du '\n' final) */
add_history(ligne);
/* ajoute la ligne à l'historique, permettant sa réutilisation avec les
 * flèches du clavier */
```

L'affichage du prompt de `fsh` est réalisé **sur sa sortie erreur**. Avec la bibliothèque `readline`, ce comportement s'obtient en indiquant `rl_outstream = stderr;` avant le premier appel à la fonction `readline()`.

2.1.5.1 Formatage du prompt

Le prompt est limité à 30 caractères (apparents, ie. sans compter les indications de couleur), et est formé des éléments suivants :

- un code de bascule de couleur, "`\033[32m`" (vert) ou "`\033[91m`" (rouge) suivant que la dernière commande exécutée a réussi ou échoué;
- entre crochets, la **valeur de retour** de la dernière commande exécutée (0 par défaut pour le premier prompt) ou "`SIG`" si elle a été interrompue par un signal;
- une autre bascule de couleur, par exemple "`\033[34m`" (bleu) ou "`\033[36m`" (cyan);
- la référence du **répertoire courant**, éventuellement tronquée (par la gauche) pour respecter la limite de 30 caractères; dans ce cas la référence commencera par trois points ("`...`");
- la bascule "`\033[00m`" (retour à la normale);
- un dollar puis un espace ("`$ `").

Pour que l'affichage s'adapte correctement à la géométrie de la fenêtre, chaque bascule de couleur doit être entourée des balises `"\001"` et `"\002"` (qui indiquent que la portion de chaîne qu'elles délimitent est formée de caractères non imprimables et doit donc être ignorée dans le calcul de la longueur du texte à afficher).

Par exemple (sans la coloration) :

```
[0]/home/titi$ cd /tmp/pas/trop/long
[0]/tmp/pas/trop/long$ cd mais/la/ca/depasse
[0]...ong/mais/la/ca/depasse$ pwd
/tmp/pas/trop/long/mais/la/ca/depasse
[0]...ong/mais/la/ca/depasse$ cd repertoire-inexistant
cd: no such file or directory: repertoire-inexistant
[1]...ong/mais/la/ca/depasse$
```

2.1.6 Sensibilité aux signaux

`fsh` ignore les signaux `SIGINT` et `SIGTERM`, contrairement aux processus exécutant des commandes externes.

Lorsque l'exécution d'une commande est interrompue par la réception d'un signal, le prompt commence par la chaîne "`[SIG]`" en lieu et place de la valeur de retour, qui est considérée comme valant 255 au cas où un appel à `exit` sans paramètre (ou un `ctrl-D`) suit.

2.2 Modalités de réalisation (et de test)

Le projet est à faire par équipes de 3 étudiants, exceptionnellement 2 si une situation particulière le justifie. La composition de chaque équipe devra être envoyée par mail à Dominique Poulalhon avec pour sujet `[SY5] équipe projet` **au plus tard le 10 novembre 2024**, avec copie à chaque membre de l'équipe.

Chaque équipe doit créer un dépôt `git` **privé** sur le `gitlab de l'UFR` **dès la constitution de l'équipe** et y donner accès en tant que `Reporter` à tous les enseignants du cours de Système : Isabelle Fagnot, Guillaume Geoffroy, Anne Micheli, Astyax Nourel et Dominique Poulalhon. Le dépôt devra contenir un fichier `AUTHORS.md` donnant la liste des membres de l'équipe (un par ligne, en précisant pour chacun, dans l'ordre : nom, prénom, numéro étudiant et pseudo(s) sur le gitlab).

En plus du code source de votre programme, vous devez fournir un `Makefile` tel que :
- l'exécution de `make` à la racine du dépôt crée (dans ce même répertoire) l'exécutable `fsh`,
- `make clean` efface tous les fichiers compilés,

ainsi qu'un fichier `ARCHITECTURE.md` expliquant la stratégie adoptée pour répondre au sujet (notamment l'architecture logicielle, les structures de données et les algorithmes implémentés).

Le projet doit s'exécuter correctement sur lulu.

Les seules interdictions strictes sont les suivantes : plagiat (d'un autre projet ou d'une source extérieure à la licence, quelle qu'elle soit), utilisation outrancière d'une IA générative de code, et utilisation des fonctions de haut niveau masquant les appels système étudiés en cours pour autre chose que l'interaction entre le shell et l'utilisateur dans le terminal (notamment la gestion de la ligne de commande avec `readline`, ou l'affichage de messages d'erreur avec `perror()`).

Pour rappel, l'emprunt de code sans citer sa source est un plagiat. L'emprunt de code en citant sa source est autorisé, mais bien sûr vous n'êtes notés que sur ce que vous avez effectivement produit. Donc si vous copiez l'intégralité de votre projet en le spécifiant clairement, vous aurez quand même 0 (mais vous éviterez une demande de convocation de la section disciplinaire).

2.3 Modalités de rendu

Le projet sera évalué en 3 phases : deux jalons intermédiaires, sans soutenance, et un rendu final avec soutenance. Les deux jalons intermédiaires seront évalués par des tests automatiques.

2.3.1 Premier jalon jeudi 28 novembre

La version à évaluer devra être spécifiée à l'aide du tag `jalon-1` sur la branche `main`.

Pour créer le tag : se placer sur le commit à étiqueter et faire :
```
git tag jalon-1
git push origin --tags
```

Points testés :

- existence du dépôt git
- présence du fichier `AUTHORS.md`
- compilation sans erreur avec `make` à la racine du dépôt
- exécution de `fsh` à la racine du dépôt
- commandes internes
- exécution des commandes externes simples (avec arguments mais sans redirections)
- conformité du prompt (y compris valeur de retour)
- boucles `for` sans option, avec corps de boucle limité à une commande simple

2.3.2 Deuxième jalon jeudi 19 décembre

La version à évaluer devra être spécifiée à l'aide du tag `jalon-2`.

Points testés : le projet complet **à l'exception** :

- des pipelines formés de strictement plus d'une commande simple
- des boucles `for` avec parallélisation (option `-p`)
- de la gestion des signaux

Par exemple, on demande que cette ligne de commande-ci fonctionne correctement :
```
for F in dir { echo $F ; if test -d $F { for G in $F -A { echo $G } } }
```

mais pas forcément celle-là :
```
cat fichier | tr a A
```

2.3.3 Rendu final début janvier

La version à évaluer devra être spécifiée à l'aide du tag `rendu-final`.

Le projet final devra être rendu à une date encore à définir mais au plus tard le 10 janvier, pour des soutenances entre le 13 et le 17 janvier.

2.3.4 Participation effective

Les projets sont des travaux de groupe, pas des travaux à répartir entre vous ("je fais le projet de Prog fonctionnelle, tu fais celui de Systèmes"). La participation effective d'un étudiant au projet de son groupe sera évaluée grâce, entre autres, aux statistiques du dépôt git et aux réponses aux questions pendant la soutenance, et les notes des étudiants d'un même groupe pourront être modulées en cas de participation déséquilibrée.

En particulier, un étudiant n'ayant aucun commit sur les aspects réellement "système" du code et incapable de répondre de manière satisfaisante sera noté "DEF".

Chapitre 3

Index des classes

3.1 Liste des classes

Liste des classes, structures, unions et interfaces avec une brève description :

Class
Structure représentant la classe dans laquelle appratient un objet . . . . . **??**
String
Classe `String` des chaines de caractères . . . . . **??**

Chapitre 4

Index des fichiers

4.1 Liste des fichiers

Liste de tous les fichiers avec une brève description :

base/Class.h
: Fichier d'en-tête contenant la définition du type Class . . . . . ??

base/main.c . . . . . ??

base/new.c
: Fichier contenant l'implémentation de toutes les méthodes génériques agissant sur des objets déclarées dans l'interface . . . . . ??

base/new.h
: Fichier d'en-tête contenant les déclarations de toutes les méthodes génériques agissant sur des objets . . . . . ??

base/String.c
: Fichier contenant l'implémentation des variables et de toutes les méthodes de l'interface . . . ??

base/String.h
: Fichier d'en-tête contenant les déclarations des variables publiques et de toutes les méthodes public de String . . . . . ??

Chapitre 5

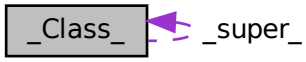
Documentation des classes

5.1 Référence de la structure _Class_

Structure représentant la classe dans laquelle appratient un objet.

`#include <Class.h>`

Graphe de collaboration de _Class_:



Attributs publics

- struct _Class_ * _super_
  Classe du parent. NULL si pas de parent.
- const char * name
  Le nom de la classe de l'objet.
- size_t size
  La taille d'allocation à la construction d'un objet.
- void *(* ctor)(void *, va_list *)
  Référence vers le constructeur d'une instance de la classe.
- void *(* dtor)(void *)
  Référence vers le destructeur d'une instance de la classe.
- char *(* to_string)(const void *)
  Référence vers la fonction `to_string` de la classe.
- int(* equals)(const void *, const void *)
  Référence vers la fonction `equals` de la classe.

5.1.1 Description détaillée

Structure représentant la classe dans laquelle appratient un objet.

Il s'agit d'une structure fournissant des informations sur la classe d'un objet créé. On y retrouve par exemple, la taille de mémoire à allouer (avec `calloc()`) lors de la construction de l'objet par le constructeur générique du fichier d'en-tête `new.h`.

Elle contient égalment des détails sur la classe, tels que son nom, la référence à la classe parente (en cas d'héritage), ainsi que les fonctions de construction et de destruction associées. Ces informations permettent au constructeur générique d'invoquer ces fonctions dynamiquement (liaison dyn.).

Cette structure peut également jouer le rôle d'une sorte d'empreinte unique pour les objets, car étant donné que chaque classe d'objets possède sa propre structure (partagée par tous les objets créés à partir de cette classe). Cela permet, par exemple, de déterminer si deux objets appartiennent à la même classe (de manière similaire au mécanisme instanceof en Java).

```
********** IMPORTANT **********
```

Important : Lors de la création d'une classe, le pointeur vers la classe des objets (le champ de type Class) doit impérativement être le premier champ déclaré dans la structure. Ce choix est essentiel, car il garantit que les fonctions génériques opérant sur les objets des classes puissent traiter n'importe quel objet comme un pointeur vers son premier élément.

Cette approche repose sur le fait qu'une structure est stockée en mémoire de manière contiguë, comme un tableau, cela permet d'accéder directement à la classe de l'objet en considérant l'objet comme un pointeur `Class*` car le champs `Class` est son premier élément.

En revanche, ce mécanisme ne fonctionne pas pour les autres champs de la structure, même le deuxième (pas comme les tableaux où on peut rajouter +1 pour aller au suivant), car ils peuvent avoir des types différents, et le compilateur peut insérer des zones de padding pour l'alignement en mémoire.

Par conséquent, cet invariant doit impérativement être respecté pour assurer le bon fonctionnement du projet.

```
**************************************************
```

5.1.2 Documentation des données membres

5.1.2.1 _super_

```
struct _Class_* _Class_::_super_
```

Classe du parent. NULL si pas de parent.

5.1.2.2 ctor

```
void*(* _Class_::ctor) (void *, va_list *)
```

Référence vers le constructeur d'une instance de la classe.

Cette fonction sera appelé dynamiquement par la fonction `obj_new()` définie dans `new.h`

5.1.2.3 dtor

```
void*(* _Class_::dtor) (void *)
```

Référence vers le destructeur d'une instance de la classe.

Cette fonction sera appelé dynamiquement par la fonction `obj_destroy()` définie dans `new.h`

5.1.2.4 equals

```
int(* _Class_::equals) (const void *, const void *)
```

Référence vers la fonction `equals` de la classe.

Cette fonction sera appelé dynamiquement par la fonction `obj_equals()` définie dans `new.h`

5.1.2.5 name

```
const char* _Class_::name
```

Le nom de la classe de l'objet.

5.1.2.6 size

```
size_t _Class_::size
```

La taille d'allocation à la construction d'un objet.

5.1.2.7 to_string

```
char*(* _Class_::to_string) (const void *)
```

Référence vers la fonction `to_string` de la classe.

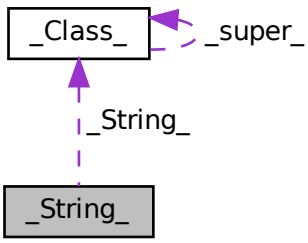
Cette fonction sera appelé dynamiquement par la fonction `obj_to_string()` définie dans `new.h`

La documentation de cette structure a été générée à partir du fichier suivant :
— base/Class.h

5.2 Référence de la structure _String_

Classe `String` des chaines de caractères.

Graphe de collaboration de _String_:



Attributs publics

— Class _String_
Classe d'un objet String.

5.2.1 Description détaillée

Classe `String` des chaines de caractères.

Il s'agit d'un boxing du type `char*` en object de type String.

Il permet nottement de les utiliser dans des structure de donnée génériques comme les listes.

Construction : obj_new(*String*, <chaine_de_caractere>);

5.2.2 Documentation des données membres

5.2.2.1 _String_

`Class _String_::_String_`

Classe d'un objet String.

La documentation de cette structure a été générée à partir du fichier suivant :
— base/String.c

Chapitre 6

Documentation des fichiers

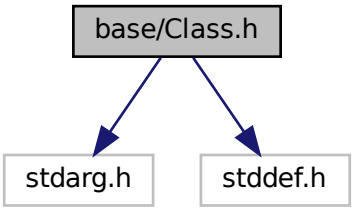
6.1 Référence du fichier AUTHORS.md

6.2 Référence du fichier base/Class.h

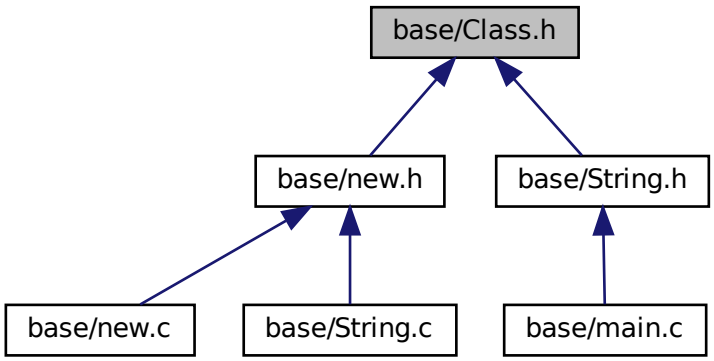
Fichier d'en-tête contenant la définition du type Class.

```
#include <stdarg.h>
#include <stddef.h>
```

Graphe des dépendances par inclusion de Class.h:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Classes

— struct _Class_
Structure représentant la classe dans laquelle appratient un objet.

Définitions de type

— typedef struct _Class_ * Class
Structure représentant la classe dans laquelle appratient un objet.

6.2.1 Description détaillée

Fichier d'en-tête contenant la définition du type Class.

6.2.2 Documentation des définitions de type

6.2.2.1 Class

```
typedef struct _Class_* Class
```

Structure représentant la classe dans laquelle appratient un objet.

Il s'agit d'une structure fournissant des informations sur la classe d'un objet créé. On y retrouve par exemple, la taille de mémoire à allouer (avec `calloc()`) lors de la construction de l'objet par le constructeur générique du fichier d'en-tête `new.h`.

Elle contient égalment des détails sur la classe, tels que son nom, la référence à la classe parente (en cas d'héritage), ainsi que les fonctions de construction et de destruction associées. Ces informations permettent au constructeur générique d'invoquer ces fonctions dynamiquement (liaison dyn.).

Cette structure peut également jouer le rôle d'une sorte d'empreinte unique pour les objets, car étant donné que chaque classe d'objets possède sa propre structure (partagée par tous les objets créés à partir de cette classe). Cela permet, par exemple, de déterminer si deux objets appartiennent à la même classe (de manière similaire au mécanisme instanceof en Java).

```
********** IMPORTANT **********
```

Important : Lors de la création d'une classe, le pointeur vers la classe des objets (le champ de type Class) doit impérativement être le premier champ déclaré dans la structure. Ce choix est essentiel, car il garantit que les fonctions génériques opérant sur les objets des classes puissent traiter n'importe quel objet comme un pointeur vers son premier élément.

Cette approche repose sur le fait qu'une structure est stockée en mémoire de manière contiguë, comme un tableau, cela permet d'accéder directement à la classe de l'objet en considérant l'objet comme un pointeur `Class*` car le champs `Class` est son premier élément.

En revanche, ce mécanisme ne fonctionne pas pour les autres champs de la structure, même le deuxième (pas comme les tableaux où on peut rajouter +1 pour aller au suivant), car ils peuvent avoir des types différents, et le compilateur peut insérer des zones de padding pour l'alignement en mémoire.

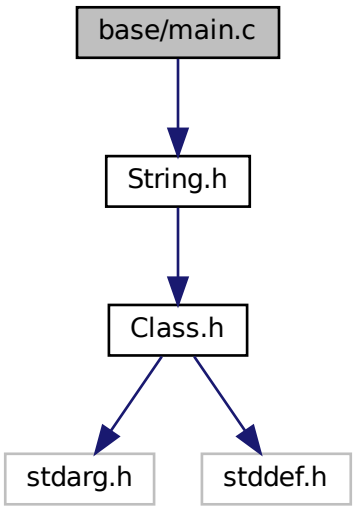
Par conséquent, cet invariant doit impérativement être respecté pour assurer le bon fonctionnement du projet.

```
**********************************************************
```

6.3 Référence du fichier base/main.c

```
#include "String.h"
```

Graphe des dépendances par inclusion de main.c:

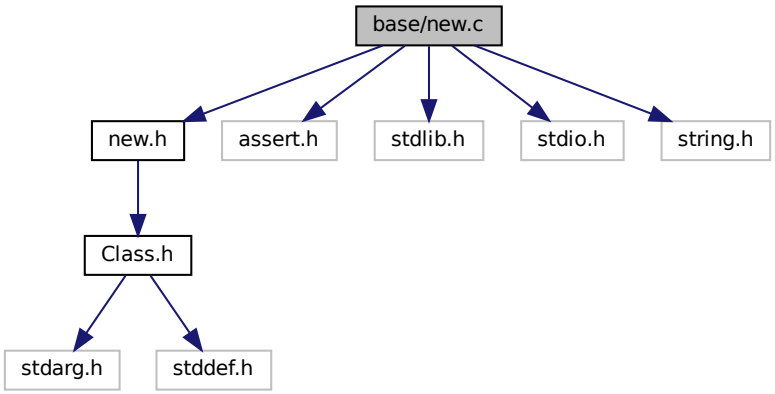


6.4 Référence du fichier base/new.c

Fichier contenant l'implémentation de toutes les méthodes génériques agissant sur des objets déclarées dans l'interface.

```
#include "new.h"
#include "assert.h"
#include "stdlib.h"
#include <stdio.h>
#include <string.h>
```

Graphe des dépendances par inclusion de new.c:



Fonctions

- void * obj_new (void *__class__,...)
  Fonction qui crée un objet à partir de sa classe.
- void obj_destroy (void *self_)
  Fonction qui détruit et desalloue l'espace mémoire alloué pour l'objet.
- char * obj_to_string (const void *self_)
  Fonction qui renvoie une chaine de caractère représentative de l'objet.
- int obj_equals (const void *self_, const void *object_)
  Fonction qui renvoie si oui ou non les deux objets sont egaux.
- int obj_instance_of (const void *object, const Class _class_)
  Fonction qui permet de vérifier si un objet est l'instance d'une classe donnée.

Variables

- char to_string_buffer [1024]
  Mémoire tampon qui doit être utilisée dans les fonctions `to_string` des objets. Cette mémmoire est généralement statique, afin d'éviter les allocations dynamiques.

6.4.1 Description détaillée

Fichier contenant l'implémentation de toutes les méthodes génériques agissant sur des objets déclarées dans l'interface.

Ces méthode sont générique pour permettre la création et la manipulation d'objets sans connaitre réelment leurs types et leurs structures.
Indispensable si on veut faire de la programmation orienté objet de manière plus efficace.

6.4.2 Documentation des fonctions

6.4.2.1 obj_destroy()

```
void obj_destroy (
            void * self_ )
```

Fonction qui détruit et desalloue l'espace mémoire alloué pour l'objet.

La fonction va appeler la fonction `dtor()` définie dans la classe de l'objet, qui va désallouer tous les éléments utilisés dans l'objet, puis après l'appel, va désallouer l'objet avec `free()`.

NB : La desallocation du pointeur vers l'objet `self_` ne doit pas se faire dans `dtor()`, cela va générer une erreur car au retour de l'appel cette fonction le fait également (il y aura donc deux `free()` sur une référence)).

Paramètres

*self_*	L'objet à détruire.

Exceptions

Abort	Le programme est interrompu si une fonction de destruction n'est pas définie dans la classe.

6.4.2.2 obj_equals()

```
int obj_equals (
            const void * self_,
            const void * object_ )
```

Fonction qui renvoie si oui ou non les deux objets sont egaux.

La fonction appelle la fonction `equals()` définie dans la classe de l'objet. Si la fonction n'est pas définie elle compare bit à bit les deux objets en mémoire. (avec la fonction `memcmp()`)

Paramètres

*self_*	L'objet 1.
*object_*	L'objet 2.

Renvoie

Si oui (1) ou non (0) objet1 et objet2 sont égaux.

6.4.2.3 obj_instance_of()

```
int obj_instance_of (
            const void * self,
            const Class _class_ )
```

Fonction qui permet de vérifier si un objet est l'instance d'une classe donnée.

Cette fonction fonctionne également sur les objets dont la classe hérite d'une autre (elle vérifie récursivement avec les ancètres).

Paramètres

*self_*	L'objet dont on souhaite vérifier l'appartenance à la classe.
*_class_*	La classe.

Renvoie

Si oui (1) ou non (0) l'objet est une instance de la classe.

6.4.2.4 obj_new()

```
void* obj_new (
            void * _class_,
             ... )
```

Fonction qui crée un objet à partir de sa classe.

La fonction alloue dynamiquement la mémoire l'objet avec calloc(), et appelle le constructeur définie dans la classe (Liaison dynamique). Si un constructeur n'est pas définie alors la fonction `obj_new()` renvoie l'objet avec tous les attribut à 0 (à cause calloc()).

Paramètres

*_class_*	La classe de l'objet à construire.
...	La liste des arguments que prend le constructeur.

Renvoie

L'objet renvoyé par le constructeur définie dans sa classe ou la structure telle qu'elle a été alloué avec `calloc()` (si il n'y avait pas de constructeur).

6.4.2.5 obj_to_string()

```
char* obj_to_string (
            const void * self_ )
```

Fonction qui renvoie une chaine de caractère représentative de l'objet.

La fonction appelle la fonction `to_string()` définie dans la classe de l'objet qui doit renvoyer une chaine de caractère.

Si la fonction n'est pas définie, `obj_to_string()` renvoie le nom de la classe suivi de l'adresse du pointeur de l'objet.

ATTENTION : NE JAMAIS RENVOYER UNE CHAINE DE CARACTERE ALLOUEE DYNAMIQUEMENT !
Il est impératif d'écrire directement dans le buffer `to_string_buffer` situé en dessous et de laisser au code appelant la responsabilité de décider s'il souhaite en faire une copie ou non.

Paramètres

self↩ _	L'objet à interroger.

Renvoie

La chaine de caractère renvoyé par `to_string`, qui est généralement un pointeur vers `to_string_`↩ `buffer` ci-dessous.

6.4.3 Documentation des variables

6.4.3.1 to_string_buffer

```
char to_string_buffer[1024]
```

Mémoire tampon qui doit être utilisée dans les fonctions `to_string` des objets. Cette mémmoire est généralement statique, afin d'éviter les allocations dynamiques.

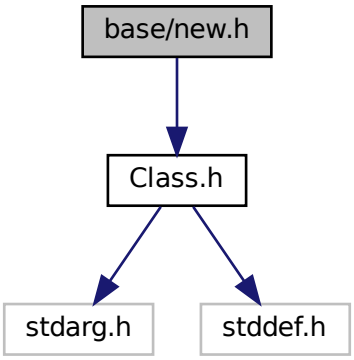
De cette façon on réduit les risques de fuites mémoire et on confie au code appelant la responsabilité de gérer, si nécessaire, une copie de la chaîne résultante.

6.5 Référence du fichier base/new.h

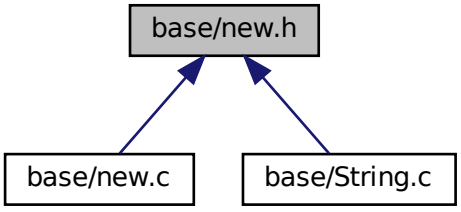
Fichier d'en-tête contenant les déclarations de toutes les méthodes génériques agissant sur des objets.

```
#include "Class.h"
```

Graphe des dépendances par inclusion de new.h:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Fonctions

- void * obj_new (void *_class_,...)
  Fonction qui crée un objet à partir de sa classe.
- void obj_destroy (void *self_)
  Fonction qui détruit et desalloue l'espace mémoire alloué pour l'objet.
- char * obj_to_string (const void *self_)
  Fonction qui renvoie une chaine de caractère représentative de l'objet.
- int obj_equals (const void *self_, const void *object_)
  Fonction qui renvoie si oui ou non les deux objets sont egaux.
- int obj_instance_of (const void *self, const Class _class_)
  Fonction qui permet de vérifier si un objet est l'instance d'une classe donnée.

Variables

- char to_string_buffer [1024]
  Mémoire tampon qui doit être utilisée dans les fonctions `to_string` des objets. Cette mémmoire est généralement statique, afin d'éviter les allocations dynamiques.

6.5.1 Description détaillée

Fichier d'en-tête contenant les déclarations de toutes les méthodes génériques agissant sur des objets.

Ce fichier contient les signatures publiques des fonctions générique agissant sur des objets, par exemple `obj_new()` qui est une fonction qui initialise un nouvel objet, à partir d'une classe.

6.5.2 Documentation des fonctions

6.5.2.1 obj_destroy()

```
void obj_destroy (
            void * self_ )
```

Fonction qui détruit et desalloue l'espace mémoire alloué pour l'objet.

La fonction va appeler la fonction `dtor()` définie dans la classe de l'objet, qui va désallouer tous les éléments utilisés dans l'objet, puis après l'appel, va désallouer l'objet avec `free()`.

NB : La desallocation du pointeur vers l'objet `self_` ne doit pas se faire dans `dtor()`, cela va générer une erreur car au retour de l'appel cette fonction le fait également (il y aura donc deux `free()` sur une référence)).

Paramètres

*self_*	L'objet à détruire.

Exceptions

Abort	Le programme est interrompu si une fonction de destruction n'est pas définie dans la classe.

6.5.2.2 obj_equals()

```
int obj_equals (
            const void * self_,
            const void * object_ )
```

Fonction qui renvoie si oui ou non les deux objets sont egaux.

La fonction appelle la fonction `equals()` définie dans la classe de l'objet. Si la fonction n'est pas définie elle compare bit à bit les deux objets en mémoire. (avec la fonction `memcmp()`)

Paramètres

*self_*	L'objet 1.
*object_*	L'objet 2.

Renvoie

Si oui (1) ou non (0) objet1 et objet2 sont égaux.

6.5.2.3 obj_instance_of()

```
int obj_instance_of (
            const void * self,
            const Class _class_ )
```

Fonction qui permet de vérifier si un objet est l'instance d'une classe donnée.

Cette fonction fonctionne également sur les objets dont la classe hérite d'une autre (elle vérifie récursivement avec les ancètres).

Paramètres

*self_*	L'objet dont on souhaite vérifier l'appartenance à la classe.
*_class_*	La classe.

Renvoie

Si oui (1) ou non (0) l'objet est une instance de la classe.

6.5.2.4 obj_new()

```
void* obj_new (
            void * _class_,
             ... )
```

Fonction qui crée un objet à partir de sa classe.

La fonction alloue dynamiquement la mémoire l'objet avec calloc(), et appelle le constructeur définie dans la classe (Liaison dynamique). Si un constructeur n'est pas définie alors la fonction `obj_new()` renvoie l'objet avec tous les attribut à 0 (à cause calloc()).

Paramètres

*_class_*	La classe de l'objet à construire.
...	La liste des arguments que prend le constructeur.

Renvoie

L'objet renvoyé par le constructeur définie dans sa classe ou la structure telle qu'elle a été alloué avec `calloc()` (si il n'y avait pas de constructeur).

6.5.2.5 obj_to_string()

```
char* obj_to_string (
            const void * self_ )
```

Fonction qui renvoie une chaine de caractère représentative de l'objet.

La fonction appelle la fonction `to_string()` définie dans la classe de l'objet qui doit renvoyer une chaine de caractère.

Si la fonction n'est pas définie, `obj_to_string()` renvoie le nom de la classe suivi de l'adresse du pointeur de l'objet.

ATTENTION : NE JAMAIS RENVOYER UNE CHAINE DE CARACTERE ALLOUEE DYNAMIQUEMENT !
Il est impératif d'écrire directement dans le buffer `to_string_buffer` situé en dessous et de laisser au code appelant la responsabilité de décider s'il souhaite en faire une copie ou non.

Paramètres

*self_*	L'objet à interroger.

Renvoie

La chaine de caractère renvoyé par `to_string`, qui est généralement un pointeur vers `to_string_buffer` ci-dessous.

6.5.3 Documentation des variables

6.5.3.1 to_string_buffer

```
char to_string_buffer[1024]  [extern]
```

Mémoire tampon qui doit être utilisée dans les fonctions `to_string` des objets. Cette mémmoire est généralement statique, afin d'éviter les allocations dynamiques.

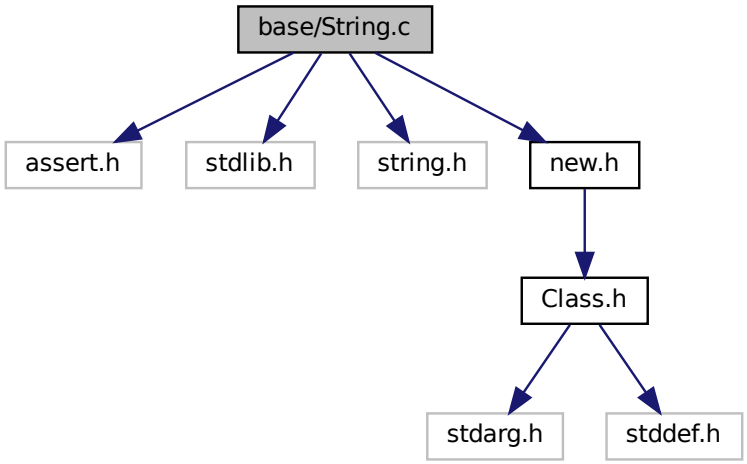
De cette façon on réduit les risques de fuites mémoire et on confie au code appelant la responsabilité de gérer, si nécessaire, une copie de la chaîne résultante.

6.6 Référence du fichier base/String.c

Fichier contenant l'implémentation des variables et de toutes les méthodes de l'interface.

```
#include <assert.h>
#include <stdlib.h>
#include <string.h>
#include "new.h"
```

Graphe des dépendances par inclusion de String.c:



Classes

— struct _String_
Classe `String` des chaines de caractères.

Définitions de type

— typedef struct _String_ * String
Classe `String` des chaines de caractères.

Variables

— Class _String_ = &__String__

6.6.1 Description détaillée

Fichier contenant l'implémentation des variables et de toutes les méthodes de l'interface.

Ce fichier contient le code sources des fonctions de l'interface `String.h` mais aussi les constructeurs, les destructeurs, etc ...

6.6.2 Documentation des définitions de type

6.6.2.1 String

```
typedef struct _String_* String
```

Classe `String` des chaines de caractères.

Il s'agit d'un boxing du type `char*` en object de type String.

Il permet nottement de les utiliser dans des structure de donnée génériques comme les listes.

Construction : obj_new(*String*, <chaine_de_caractere>);

6.6.3 Documentation des variables

6.6.3.1 _String_

```
Class _String_ = &__String__
```

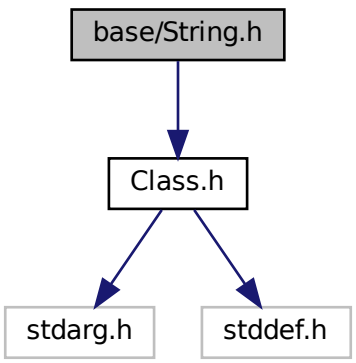
Classe de tous les objets de type String.

6.7 Référence du fichier base/String.h

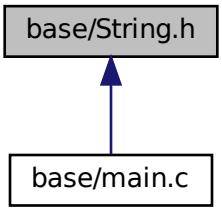
Fichier d'en-tête contenant les déclarations des variables publiques et de toutes les méthodes public de String.

```
#include "Class.h"
```

Graphe des dépendances par inclusion de String.h:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



6.7.1 Description détaillée

Fichier d'en-tête contenant les déclarations des variables publiques et de toutes les méthodes public de String.

6.8 Référence du fichier projet.md